

Program: 6 Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests.

Create a Maven Project in Eclipse (Skip Archetype)

1. Open Eclipse IDE

Navigate to:

File → New → Project → Maven → Maven Project → Click Next

2. In the wizard:

- Uncheck "Create a simple project (skip archetype selection)" → ☒ Check this box
- Click Next

3. Fill in:

- **Group Id:** com.example
- **Artifact Id:** helloworld-ci
- **Version:** default
- Packaging: jar
→ Click **Finish**

4. Eclipse creates a blank Maven structure:

- Add new class: src/main/java/com/example/App.java

```
package com.example;

public class App {
    public static void main(String[] args) {
        System.out.println("Hello, Continuous Integration!");
    }
}
```

5. Build and Test Locally (Optional Pre-check)

- Right-click the project → **Run As** → **Maven install**.
- Verify build success in console.

Push Code to GitHub Using Eclipse IDE:

1. Share Maven Project to Git

- Right-click on your Maven project → **Team** → **Share Project**
- Select **Git** → **Next**
- Choose or create a local repository → **Finish**

2. Commit Your Project

- Right-click project → **Team** → **Commit**
- Enter commit message (e.g., *"Initial Commit"*)
- Stage all files (select all checkboxes) → **Click Commit**

3. Create GitHub Repository (via browser)

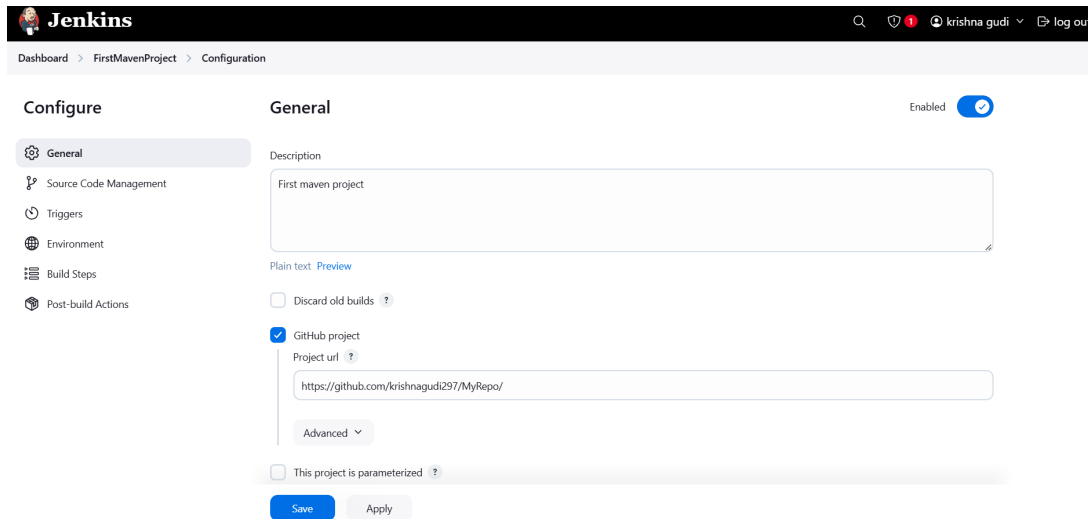
- Go to <https://github.com> → **Click + New repository**
- Set repo name (e.g., maven-jenkins-ci)
- Do not add README or .gitignore → **Click Create repository**

4. Push to GitHub from Eclipse

- In Eclipse, open Git Repositories view (Window → Show View → Other → Git → Git Repositories)
- Right-click your local repo → **Remote** → **Configure Push to Upstream**
- Enter GitHub URL (e.g., <https://github.com/yourusername/maven-jenkins-ci.git>)
- Click Next → **Select master or main as source & destination** → **Add Spec**
- Click Finish → **Right-click project** → **Team** → **Push Branch 'main'**

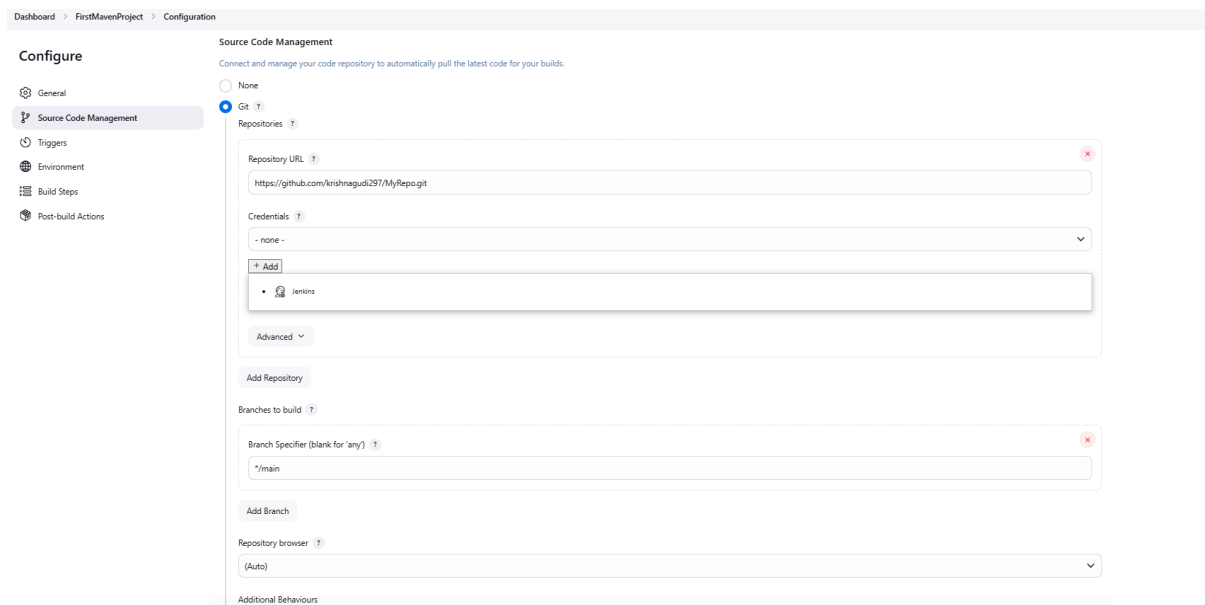
Integrate Jenkins with GitHub and Run Maven Build:

STEP 1: From the Jenkins dashboard, click “**New Item**”, enter a job name, select “**Freestyle project**”, and click OK to create a new Jenkins job for your Maven project.



The screenshot shows the Jenkins Configuration page for a project named 'FirstMavenProject'. The 'General' tab is selected in the left sidebar. The 'Description' field contains 'First maven project'. The 'Discard old builds' checkbox is unchecked. The 'GitHub project' checkbox is checked, and the 'Project url' field contains 'https://github.com/krishnagudi297/MyRepo/'. The 'Advanced' dropdown is set to 'Advanced'. The 'This project is parameterized' checkbox is unchecked. The 'Save' button is highlighted.

STEP 2: In the project configuration page, under “**Source Code Management**”, select **Git**, then provide the GitHub repository URL and specify the branch (e.g., */main).



The screenshot shows the Jenkins Configuration page for the 'Source Code Management' tab. The 'Git' radio button is selected. The 'Repository URL' field contains 'https://github.com/krishnagudi297/MyRepo.git'. The 'Credentials' dropdown is set to 'none'. The 'Add Repository' button is highlighted. The 'Branches to build' section shows the 'Branch Specifier (blank for 'any')' field set to '*/main'. The 'Add Branch' button is highlighted. The 'Repository browser' dropdown is set to 'Auto'.

STEP 3: Scroll down & enable “**GitHub hook trigger for GITScm polling**” to automatically start builds when code is pushed to GitHub. In the Environment section, check “**Delete workspace before build starts**” to ensure a clean build each time.

Dashboard > FirstMavenProject > Configuration

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Build Steps
- Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

- ☒ Delete workspace before build starts

Advanced ▾

- ☐ Use secret text(s) or file(s) ?
- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published build scans
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?

STEP 4: Since the **pom.xml** is inside a subdirectory (e.g., MyMaven), update the **POM field** with the correct relative path like **MyMaven/pom.xml** to ensure Jenkins builds the right file.

Dashboard > FirstMavenProject > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps**
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Invoke top-level Maven targets ?

Goals

clean test ▾

Advanced ▴ Edited

POM ?

MyMaven/pom.xml

Properties ?

JVM Options ?

☐ Inject build variables ?

☐ Use private Maven repository

Settings file ?

Use default maven settings ▾ ?

Global Settings file ?

Use default maven global settings ▾ ?

Save Apply